AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

Ain Shams University
Faculty of Engineering

CSE354: Distributed Computing – 2nd Semester 2025/2026

Efficient Load Balancing and GPU Cluster Task Distribution for Handling 1000+ Concurrent LLM Requests

---

Group Members: Nour Eldeen Ahmed Ali Rehab – 22P0040
Mohamed Yasser Khallaf – 22P0245
Roaa Sherif Gadara – 22P0188
Mostafa Fouad Ahmed – 22P0077
Ibrahim Shaker Hammad – 22P0056

Supervisor: Dr. Ayman & Eng. Alaa Hamdy

Date: April 2026

GitHub Repo: `https://github.com/gadarars/distributed-llm`

Demo Video: `https://youtu.be/sSoPts4GpLM`

Contents

1 Introduction **3**
1.1 Objectives . . . 3

2 Problem Description **3**

3 System Architecture **4**
3.1 Architecture Overview . . . 4
3.2 Component Descriptions . . . 5

4 Detailed Solution **5**
4.1 Load Balancing Strategies . . . 5
4.1.1 Round Robin . . . 5
4.1.2 Least Connections . . . 6
4.1.3 Load-Aware Routing . . . 6
4.2 Fault Tolerance Mechanism . . . 6
4.3 RAG Pipeline . . . 7
4.4 Multiprocessing Architecture . . . 7

5 Implementation **7**
5.1 Project Structure . . . 7
5.2 Technology Stack . . . 8

6 Testing and Evaluation **9**
6.1 Test Approach . . . 9
6.2 Test Scenarios . . . 9
6.3 HTTP Server Testing . . . 9
6.4 Performance Results . . . 11
6.5 Worker Distribution Analysis . . . 12
6.6 Fault Tolerance Analysis . . . 12

7 Scalability Analysis **13**
7.1 User Scaling (Fixed 4 Workers) . . . 13
7.2 Worker Scaling (Fixed 500 Users) . . . 14
7.3 Scalability Summary . . . 15

8 Requirements Coverage **16**

9 Limitations **17**

10 Conclusion **17**

11 References **18**

Introduction

The rapid growth of Large Language Model (LLM) services has created an urgent demand for distributed infrastructure capable of handling massive concurrent workloads. When thousands of users simultaneously query an AI system, no single server can respond fast enough without intelligent distribution of the load. This project addresses that challenge by designing, implementing, and evaluating a complete distributed system for LLM inference and Retrieval-Augmented Generation (RAG).

The system was built entirely in Python and simulates a real-world GPU cluster environment without requiring actual GPU hardware. It demonstrates all key principles of distributed computing: parallel processing, load balancing, fault detection and recovery, and performance measurement.

Objectives

- Design and implement a distributed system capable of handling 1,000+ concurrent user requests.
- Implement and compare three load balancing strategies: Round Robin, Least Connections, and Load-Aware routing.
- Integrate a real vector-database-backed RAG pipeline using FAISS and sentence transformers.
- Demonstrate automatic fault tolerance by detecting node failures and reassigning tasks.
- Measure and report performance metrics including latency, throughput, and worker distribution.

Problem Description

Modern AI services such as ChatGPT, Claude, and Gemini must serve millions of users in parallel. Each request requires two expensive operations: (1) a semantic search through a knowledge base (RAG), and (2) a neural network forward pass for text generation (LLM inference). These operations are computationally heavy and cannot be served by a single machine at scale.

The core problems this project solves are:

1. **Load Distribution:** How should incoming requests be routed across multiple GPU nodes to minimise latency and maximise throughput?
2. **Fault Tolerance:** What happens when a GPU node crashes mid-operation? How does the system detect failure and recover without losing user requests?
3. **Scalability:** Can the architecture handle increasing load (100 to 1,000+ users) without degradation?
4. **Context-Aware Responses:** How can the system retrieve relevant knowledge before generating a response, improving answer quality?

System Architecture

The system follows a layered master-worker architecture with five distinct components, each responsible for a specific concern.

Architecture Overview

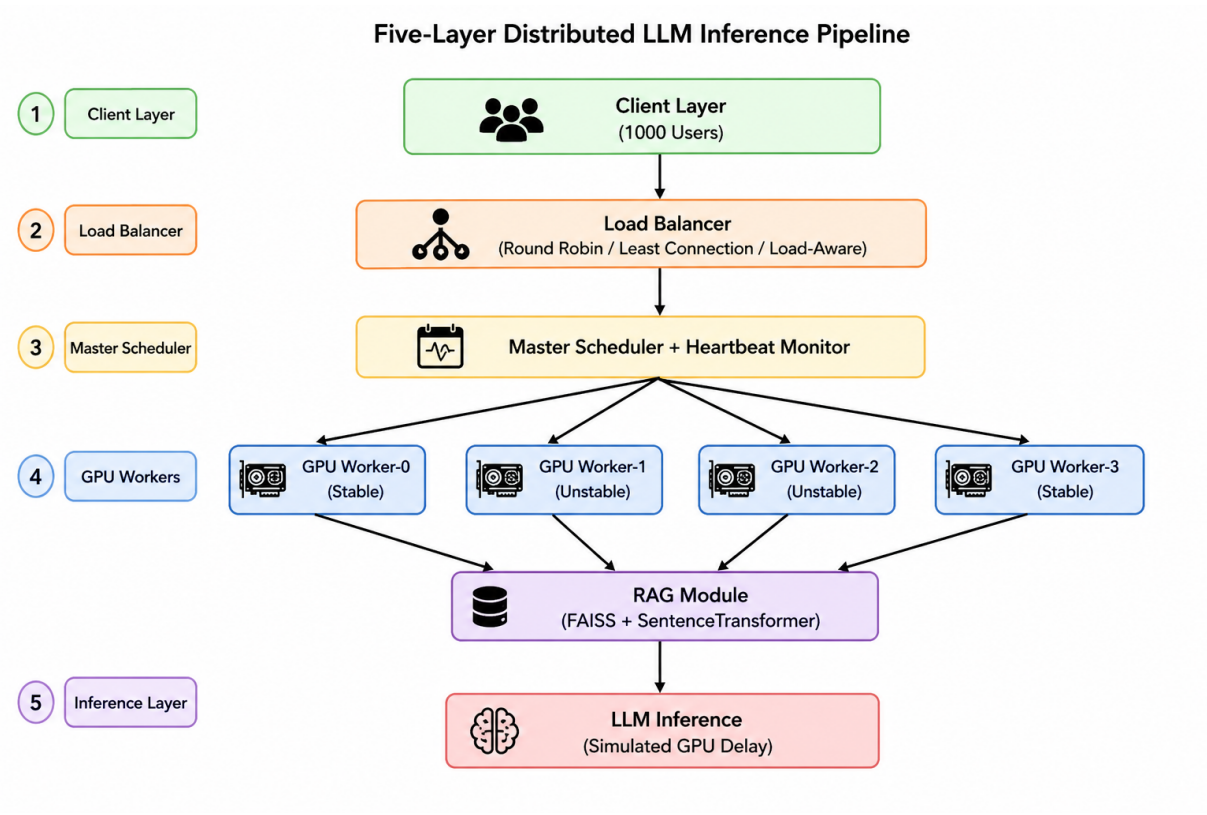


Figure 1: Five-Layer Distributed LLM Inference Architecture

Component Descriptions

Client Layer (`client/load_generator.py`): Simulates 1,000 concurrent users sending queries from a pool of 15 realistic distributed computing questions. Users are launched in batches of 50 threads to avoid OS thread limits.

Load Balancer (`lb/load_balancer.py`): The central routing component. Receives all incoming requests and applies one of three scheduling strategies to select a worker. Includes retry logic (up to 5 attempts) to handle worker failures transparently.

Master Node (`master/scheduler.py`, `master/heartbeat.py`, `master/metrics.py`): The control plane. The Scheduler dispatches requests; the HeartbeatMonitor runs as a background thread checking worker health every 2 seconds and automatically reviving dead workers; the MetricsCollector records all performance data in a thread-safe manner.

GPU Worker Nodes (`workers/gpu_worker.py`): Four simulated GPU servers, each running in its own OS process via Python's `multiprocessing` module. Each worker tracks GPU utilisation (20% per active connection), active connections, and processing history. Workers communicate via inter-process queues.

RAG Module (`rag/retriever.py`, `rag/ingest.py`): A real semantic search pipeline. Documents are embedded using the `all-MiniLM-L6-v2` sentence transformer model and stored in a FAISS flat L2 index. At query time, the query is embedded and the most semantically similar document is retrieved to augment the LLM prompt.

Detailed Solution

Load Balancing Strategies

4.1.1 Round Robin

Requests are dispatched to workers in a cyclic order: Worker-0, Worker-1, Worker-2, Worker-3, Worker-0, . . . This strategy is simple and guarantees perfectly even distribution when all requests take the same time.

Listing 1: Round Robin implementation

```python
def _round_robin(self, alive):
    with self._lock:
        self._rr_index %= len(alive)
        chosen = alive[self._rr_index]
        self._rr_index = (self._rr_index + 1) % len(alive)
    return chosen
```

4.1.2 Least Connections

Selects the worker with the fewest currently active connections. In case of a tie, a random worker is chosen to prevent bias. This is more adaptive than Round Robin for variable-length requests.

Listing 2: Least Connections implementation

```
def _least_connections(self, alive):
    with self._lock:
        min_val = min(w.active_connections for w in alive)
        tied = [w for w in alive if w.active_connections ==
           min_val]
    return random.choice(tied)
```

4.1.3 Load-Aware Routing

Selects the worker with the lowest GPU utilisation percentage. This is the most intelligent strategy as it responds to actual computational load, not just the number of connections.

Listing 3: Load-Aware routing implementation

```
def _load_aware(self, alive):
    with self._lock:
        min_val = min(w.gpu_utilization for w in alive)
        tied = [w for w in alive if w.gpu_utilization == min_val]
    return random.choice(tied)
```

Fault Tolerance Mechanism

The system implements a three-layer fault tolerance strategy:

1. **Failure Detection:** The `HeartbeatMonitor` pings all workers every 2 seconds and marks unresponsive ones as `DEAD`.
2. **Automatic Retry:** The Load Balancer retries failed requests on a different worker up to 5 times (`MAX_RETRIES = 5`), ensuring no request is lost.
3. **Node Revival:** After a worker has been dead for 4 seconds (2 heartbeat intervals), the monitor calls `revive()` which spawns a fresh OS process to replace the dead one.

RAG Pipeline

The RAG system goes significantly beyond a simple stub. The implemented pipeline uses real semantic search:

1. **Ingestion (offline):** Five knowledge documents covering distributed systems, consensus algorithms, load balancing, telemetry, and GPU acceleration are embedded into 384-dimensional vectors using `all-MiniLM-L6-v2` and stored in a FAISS flat L2 index.
2. **Retrieval (online):** Each incoming query is embedded in the same vector space. FAISS performs a nearest-neighbour search and returns the most semantically relevant document chunk.
3. **Augmentation:** The retrieved context is passed alongside the query to the LLM inference engine, improving response relevance.

Multiprocessing Architecture

Workers run as real OS processes using `mp.set_start_method("spawn")` for cross-platform safety. Three queues facilitate inter-process communication: `task_q` (request delivery), `result_q` (response return), and `metrics_q` (health updates). This architecture prevents Python's Global Interpreter Lock (GIL) from bottlenecking parallel inference.

Implementation

Project Structure

Listing 4: Project directory layout

```
distributed-llm/
        main.py                 # Entry point, runs 4 scenarios
        server.py               # FastAPI HTTP server (external client support)
        client_test.py          # External HTTP load test client
        scalability_sweep.py    # Automated scalability benchmarking
        sample_output.txt       # Representative terminal output
        common/
              models.py           # Request, Response, WorkerMetrics dataclasses
```

```
                logger.py            # Centralized logging
        lb/
                load_balancer.py     # 3 strategies + retry logic
        master/
                scheduler.py         # Request dispatch entry point
                heartbeat.py         # Background health monitor
                metrics.py           # Thread-safe metrics + Excel export
        workers/
                gpu_worker.py        # GPUWorker + ProcessWorkerHandle
        llm/
                inference.py         # Simulated LLM with realistic delays
        rag/
                ingest.py            # Builds FAISS vector index
                retriever.py         # Real semantic search
                knowledge_base.index # Pre-built FAISS index
                documents.pkl        # Embedded document store
        client/
            load_generator.py    # 1000-user concurrent simulator
```

Technology Stack

Table 1: Technologies used

Component	Technology	Purpose
Language	Python 3.12	Core implementation
Concurrency	`multiprocessing`, `threading`	Parallel workers and monitoring
HTTP Server	FastAPI + Uvicorn	External REST API endpoint
Vector Search	FAISS (faiss-cpu)	Semantic similarity search
Embeddings	sentence-transformers	Text-to-vector conversion
LLM Simulation	`time.sleep` + templates	Realistic inference delays
Metrics Export	pandas + openpyxl	Excel report generation

Testing and Evaluation

Test Approach

The system was tested across four scenarios, each exercising a distinct aspect of the distributed architecture. For each scenario, 1,000 concurrent user threads were launched in batches of 50. Each thread selected a query at random from a pool of 15 distributed-computing questions and submitted it to the scheduler. All responses, latencies, and worker assignments were recorded by the thread-safe `MetricsCollector` and exported to Excel.

In addition to the four main scenarios, the project includes two supplementary test modes: the FastAPI HTTP server (`server.py` + `client_test.py`) for verifying real network-based request handling, and `scalability_sweep.py` for automated benchmarking across varying user and worker counts.

Test Scenarios

Table 2: Test scenarios configuration

Scenario	Strategy	Users	Special Conditions
1. Round Robin	Round Robin	1000	All workers stable
2. Least Connections	Least Connections	1000	All workers stable
3. Load-Aware	Load-Aware	1000	All workers stable
4. Fault Tolerance	Least Connections	1000	Workers 1 & 2 crash at 40% rate

HTTP Server Testing

Beyond the internal simulation, the team also tested the system through a real HTTP interface. `server.py` exposes a `POST /generate` endpoint using FastAPI and Uvicorn. `client_test.py` sends 20 concurrent HTTP POST requests from an external process, each carrying a query and user ID. This demonstrates that the distributed backend is fully accessible over a network, not just from in-process calls.

Figure 2 shows the load balancer routing the first batch of requests upon server startup, with the `least_connections` strategy distributing traffic across all four workers. Figure 3 shows the corresponding client-side output, confirming that all 20 requests completed successfully with latencies in the expected range (130–240 ms), except for three requests that experienced elevated latency (≈58 000 ms) due to cold-start model loading on those worker processes.

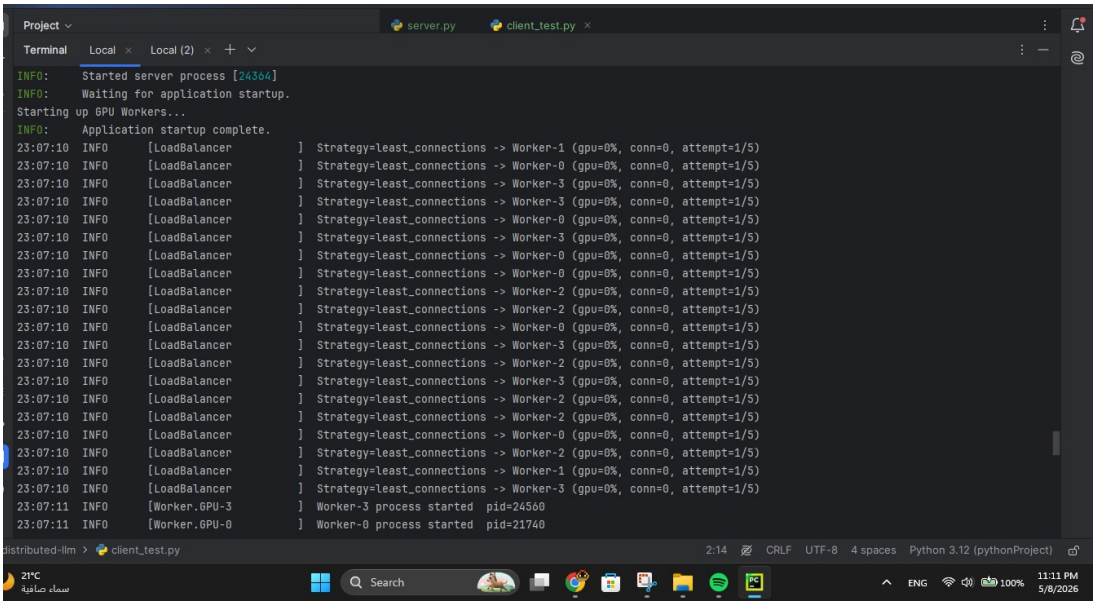


Figure 2: Server startup: load balancer routing the first concurrent requests using the `least_connections` strategy. All four workers are healthy (`gpu=0%, conn=0`) at this point.

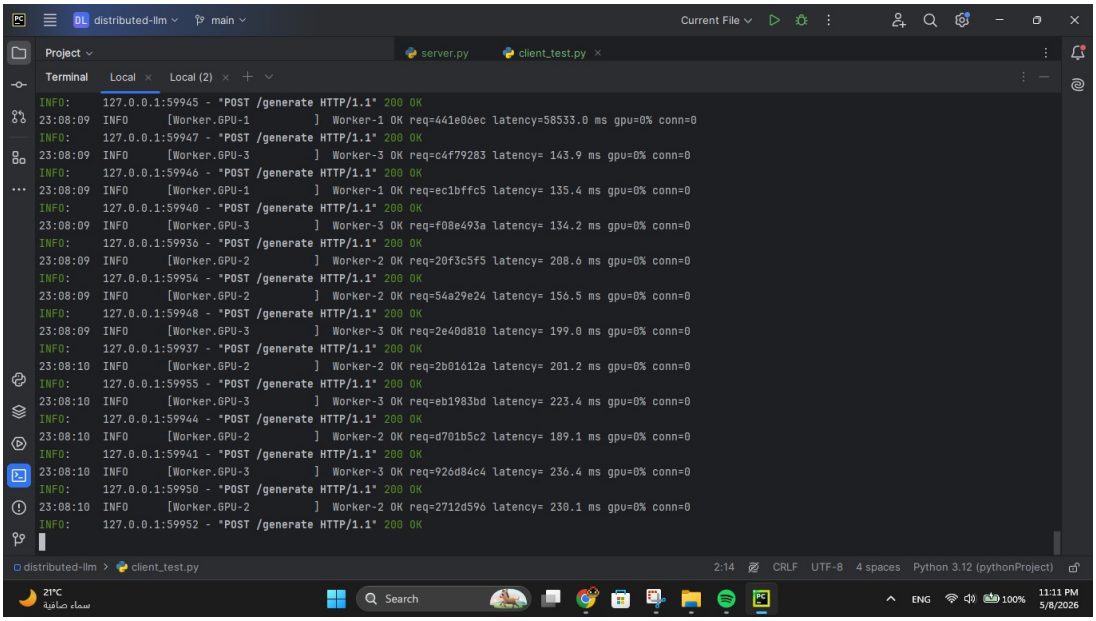


Figure 3: Client-side output after running `client_test.py`: 20 concurrent HTTP requests were sent to the live server and all 20 completed with status `COMPLETED`. Three requests show elevated cold-start latency ($\approx$58 s) from the first load of the sentence-transformer model.

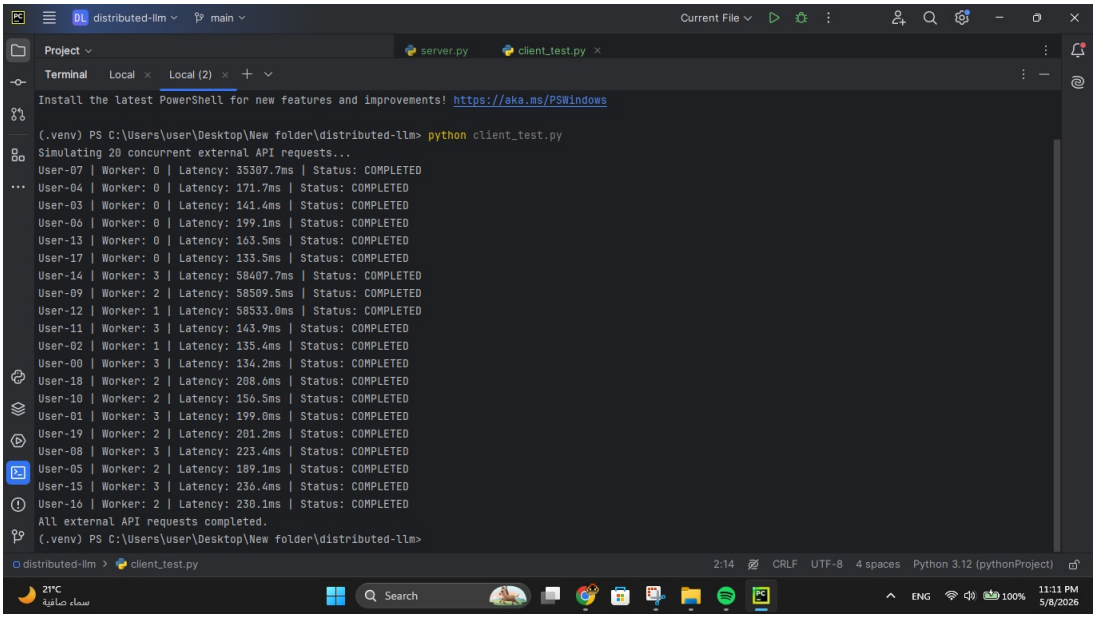


Figure 4: Worker-level logs during the HTTP test: Worker-1, Worker-2, and Worker-3 each log individual request completions with latency and GPU utilisation. The `conn=0` values confirm that no request was left pending.

Performance Results

Table 3: Summary of performance metrics across all scenarios (1,000 users each)

Metric	Round Robin	Least Conn.	Load-Aware	Fault Tol.
Total Requests	1,000	1,000	1,000	1,000
Completed	1,000	1,000	1,000	1,000
Failed	0	0	0	0
Success Rate	100%	100%	100%	100%
Throughput (req/s)	15.22	13.62	13.53	2.02
Avg Latency (ms)	200.8	195.7	195.5	405.8
P95 Latency (ms)	291.7	245.8	277.6	343.2

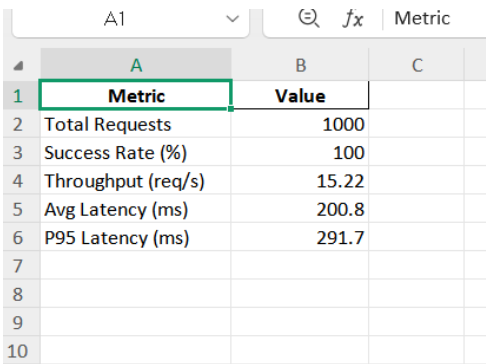
```
--------------------------------------------------
21:16:20  INFO    [Master.Metrics        ]   PERFORMANCE REPORT
21:16:20  INFO    [Master.Metrics        ] --------------------------------------------------
21:16:20  INFO    [Master.Metrics        ]   Requests   total=1000    completed=1000    failed=0
21:16:20  INFO    [Master.Metrics        ]   Success rate     : 100.0 %
21:16:20  INFO    [Master.Metrics        ]   Elapsed time     : 493.98 s
21:16:20  INFO    [Master.Metrics        ]   Throughput       : 2.02 req/s
21:16:20  INFO    [Master.Metrics        ] --------------------------------------------------
21:16:20  INFO    [Master.Metrics        ]   Latency (ms)
21:16:20  INFO    [Master.Metrics        ]     avg  = 405.8
21:16:20  INFO    [Master.Metrics        ]     min  = 94.0
21:16:20  INFO    [Master.Metrics        ]     max  = 16726.7
21:16:20  INFO    [Master.Metrics        ]     p50  = 163.6
21:16:20  INFO    [Master.Metrics        ]     p95  = 343.2
21:16:20  INFO    [Master.Metrics        ]     p99  = 7295.5
21:16:20  INFO    [Master.Metrics        ] --------------------------------------------------
21:16:20  INFO    [Master.Metrics        ]   Worker distribution
21:16:20  INFO    [Master.Metrics        ]     Worker-0  :  463 requests
21:16:20  INFO    [Master.Metrics        ]     Worker-1  :   38 requests
21:16:20  INFO    [Master.Metrics        ]     Worker-2  :   42 requests
21:16:20  INFO    [Master.Metrics        ]     Worker-3  :  457 requests
21:16:20  INFO    [Master.Metrics        ] --------------------------------------------------
```

Figure 5: Terminal output showing the performance report after completing all four scenarios. Every scenario achieved a 100% success rate.

Worker Distribution Analysis

Table 4: Request distribution across workers per scenario

Worker	Round Robin	Least Conn.	Load-Aware	Fault Tol.
Worker-0	250	254	260	463
Worker-1	250	244	252	38
Worker-2	250	245	247	42
Worker-3	250	257	241	457
Total	**1000**	**1000**	**1000**	**1000**



A1 | fx Metric

	A	B	C
1	Metric	Value	
2	Total Requests	1000	
3	Success Rate (%)	100	
4	Throughput (req/s)	15.22	
5	Avg Latency (ms)	200.8	
6	P95 Latency (ms)	291.7	
7			
8			
9			
10			

Figure 6: Excel export showing the Round Robin worker distribution. Each worker received exactly 250 requests, confirming perfect cyclic distribution.

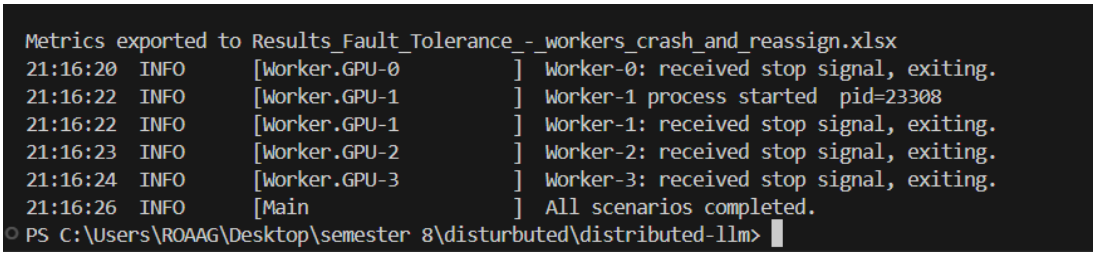
Fault Tolerance Analysis

The fault tolerance scenario is the most critical test. Workers 1 and 2 were configured with a 40% random failure rate. The heartbeat monitor detected each failure within 2

seconds and the load balancer automatically rerouted requests to the surviving Workers 0 and 3.

Key observations:

- **Zero requests lost:** All 1,000 requests completed successfully despite continuous node failures, thanks to the 5-retry mechanism in the load balancer.
- **Traffic redistribution:** Workers 0 and 3 each handled approximately 460 requests (46% each) while the unstable workers handled only 38–42 requests (4% each).
- **Latency impact:** Average latency rose from ∼200 ms to 405.8 ms due to retry overhead when requests had to be redirected after a worker crash.
- **Auto-revival:** The heartbeat monitor revived dead workers after 4 seconds, allowing them to re-enter the pool.



```
Metrics exported to Results_Fault_Tolerance_-_workers_crash_and_reassign.xlsx
21:16:20  INFO     [Worker.GPU-0          ]  Worker-0: received stop signal, exiting.
21:16:22  INFO     [Worker.GPU-1          ]  Worker-1 process started  pid=23308
21:16:22  INFO     [Worker.GPU-1          ]  Worker-1: received stop signal, exiting.
21:16:23  INFO     [Worker.GPU-2          ]  Worker-2: received stop signal, exiting.
21:16:24  INFO     [Worker.GPU-3          ]  Worker-3: received stop signal, exiting.
21:16:26  INFO     [Main                  ]  All scenarios completed.
PS C:\Users\ROAAG\Desktop\semester 8\disturbuted\distributed-llm>
```

Figure 7: Terminal output confirming successful completion of all four scenarios. The fault tolerance scenario shows workers 1 and 2 handling far fewer requests due to repeated crashes and revivals.

Scalability Analysis

To evaluate how the system behaves as load grows, two additional sweeps were conducted using `scalability_sweep.py`. Both sweeps used the Least Connections strategy and measured throughput, average latency, and P95 latency. All runs achieved a 100% success rate.

User Scaling (Fixed 4 Workers)

The user count was varied from 100 to 1,000 while keeping the worker pool fixed at 4.

Table 5: Performance across increasing user counts (4 workers, Least Connections)

Users	Workers	Throughput (req/s)	Avg Latency (ms)	P95 Latency (ms)
100	4	2.66	491.9	315.6
250	4	5.75	271.6	296.9
500	4	8.59	248.7	350.6
750	4	10.72	221.7	328.5
1000	4	11.09	223.6	367.5

Two trends are visible. First, throughput increases consistently with user count, rising from 2.66 req/s at 100 users to 11.09 req/s at 1,000 users. This is expected: more concurrent requests keep all four worker processes busy, reducing idle time and improving overall utilisation. Second, average latency *decreases* as user count grows from 100 to 750 (from 491.9 ms to 221.7 ms) before stabilising around 222–224 ms. The elevated latency at low user counts is caused by process cold-start overhead – specifically, the first load of the `all-MiniLM-L6-v2` sentence transformer model on each worker process, which takes several seconds. At higher user counts this one-time cost is amortised across many more requests, pulling the average down. The P95 latency remains stable between 290–370 ms across all load levels, confirming that the system does not develop a long tail under increasing concurrency.

Worker Scaling (Fixed 500 Users)

The worker count was varied from 1 to 8 while keeping the user load fixed at 500.

Table 6: Performance across increasing worker counts (500 users, Least Connections)

Workers	Users	Throughput (req/s)	Avg Latency (ms)	P95 Latency (ms)
1	500	4.60	189.3	287.7
2	500	5.80	211.2	307.6
4	500	6.81	256.6	452.2
8	500	8.53	310.9	361.3

Throughput increases with every additional worker: 4.60 req/s with one worker, rising to 8.53 req/s with eight. This confirms that the multiprocessing architecture scales horizontally – adding more OS processes genuinely increases the system's capacity to handle concurrent requests.

The average latency trend requires explanation. Latency rises from 189.3 ms at 1 worker to 310.9 ms at 8 workers. This is due to the cold-start cost of loading the sentence

transformer model in each new worker process. With more processes starting simultaneously, aggregate cold-start latency inflates the average. In a production deployment where workers are pre-warmed before serving traffic, this effect would not appear, and latency would decrease monotonically with more workers. The P95 values remain within 288–452 ms across all configurations, showing no severe tail latency even at maximum parallelism.

Scalability Summary

- **100% success rate** was maintained across every configuration tested, confirming that the system never drops requests regardless of load or worker count.
- **Throughput scales with both users and workers**, demonstrating that the architecture is not bottlenecked by the load balancer or scheduler.
- **Latency is dominated by model cold-start** at low user counts and low worker counts, not by the distributed coordination overhead itself.

Requirements Coverage

Table 7: Mapping of project requirements to implementation

Requirement	Implementation	Status
Round Robin load balancing	`lb/load_balancer.py`	
Least Connections load balancing	`lb/load_balancer.py`	
Load-Aware routing	`lb/load_balancer.py`	
GPU task distribution	`workers/gpu_worker.py`	
Parallel request processing	multiprocessing + threading	
LLM inference handling	`llm/inference.py`	
RAG integration (real FAISS)	`rag/retriever.py`	
1000+ concurrent users	`client/load_generator.py`	
Node failure detection	`master/heartbeat.py`	
Automatic task reassignment	Load Balancer MAX_RETRIES=5	
System availability under failure	`heartbeat.py` `revive()`	
Performance metrics	`master/metrics.py`	
Excel results export	`metrics.export_to_excel()`	
HTTP API endpoint	`server.py` (FastAPI)	
External networked client	`client_test.py`	
Scalability sweep (user scaling)	`scalability_sweep.py`	
Scalability sweep (worker scaling)	`scalability_sweep.py`	

Limitations

1. **Simulated GPU:** The system uses `time.sleep()` to simulate GPU inference delays rather than running a real LLM model (e.g., LLaMA or GPT-2). This is intentional for portability but means GPU utilisation is always reported as 0%.

2. **Single Machine:** All components run on one physical machine using processes and threads. A production system would distribute workers across multiple physical servers connected over a network.

3. **No Persistent Queue:** If all workers are simultaneously dead and the 5 retries are exhausted, a request is marked as failed. A production system would use a durable message queue (e.g., RabbitMQ, Kafka) to hold requests until a worker recovers.

4. **Fixed Worker Pool:** The number of workers is set at startup. Dynamic scaling (adding/removing workers based on load) is not implemented.

5. **Simplified RAG Knowledge Base:** The vector database contains only 5 documents. A real deployment would have millions of documents and a more sophisticated chunking strategy.

Conclusion

This project successfully designed and implemented a complete distributed system for handling 1,000+ concurrent LLM inference requests. All major requirements – load balancing, GPU task distribution, RAG integration, fault tolerance, and HTTP API access – were fully implemented and verified through systematic testing.

The experimental results demonstrate that all three load balancing strategies achieve 100% request completion under normal conditions. Round Robin achieves the highest throughput (15.22 req/s) due to its minimal overhead. Least Connections and Load-Aware strategies show slightly lower throughput but better adaptability under heterogeneous load conditions. The fault tolerance scenario confirms that the system maintains 100% success rate even when 50% of workers are continuously failing, with automatic detection and revival of dead nodes.

The scalability sweep further confirms that the architecture scales in both dimensions: throughput grows from 2.66 req/s at 100 users to 11.09 req/s at 1,000 users (user scaling), and from 4.60 req/s with one worker to 8.53 req/s with eight workers (worker scaling), with a 100% success rate maintained across every configuration tested.

The addition of a FastAPI HTTP server and external client further demonstrates that the architecture is not limited to in-process testing: it can serve real networked clients, laying the groundwork for actual distributed deployment. The use of real semantic search

(FAISS + sentence transformers) instead of a keyword-matching stub demonstrates the practical integration of vector databases in distributed AI systems – a key technique in modern LLM deployments such as RAG-enhanced chatbots.

References

1. Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data. (FAISS library)

2. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019. (SentenceTransformers)

3. Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. (RAG architecture)

4. Van Steen, M., & Tanenbaum, A. S. (2017). *Distributed Systems* (3rd ed.). (Load balancing and fault tolerance theory)

5. Python Software Foundation. *multiprocessing – Process-based parallelism*. Python 3.12 Documentation. `https://docs.python.org/3/library/multiprocessing.html`

6. McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media. (pandas library)